

3.3 Xacro 进阶建模

所属阶段： 第三阶段 · 坐标变换与机器人建模

讲次数量： 6 节

预计时长： ~50 分钟

前置依赖： 3.1 TF 坐标变换与数学基础、3.2 URDF 基础

学习目标

完成本节后，学员将能够：

- 解释 Xacro 相对于纯 URDF 的优势，并将现有 URDF 迁移为 Xacro 格式
- 使用 Xacro Properties 定义尺寸、颜色、质量等可复用变量
- 编写 Xacro Macros 封装重复的 Link/Joint 结构
- 使用 Xacro Include 将机器人模型拆分为多文件模块化组织
- 使用 `xacro` 命令将 `.xacro` 文件展开为 URDF，并集成到 ROS 2 构建流程
- 独立完成 Xacro Properties 练习，为后续 Gazebo 仿真建模打下基础

讲次 1：将 URDF 迁移为 Xacro

知识点

在 3.2 中，我们编写了纯 URDF 文件来描述机器人。当模型只有两三个 Link 时尚可维护，但随着轮组、传感器、夹爪等组件增加，URDF 会迅速膨胀——大量重复的 `<link>` / `<joint>` 块、硬编码的数字散落各处，改一个轮子半径就要全文搜索替换。

Xacro (XML Macros) 是 ROS 官方推荐的机器人描述预处理语言。它在 URDF 之上增加了**变量、数学运算、条件判断和宏函数**，最终由 `xacro` 工具展开为标准的 URDF 文件。

Xacro 与 URDF 的关系

开发阶段

编写 .xacro 文件



定义 Properties / Macros / Include



colcon build 安装到 share/

运行阶段

xacro 展开



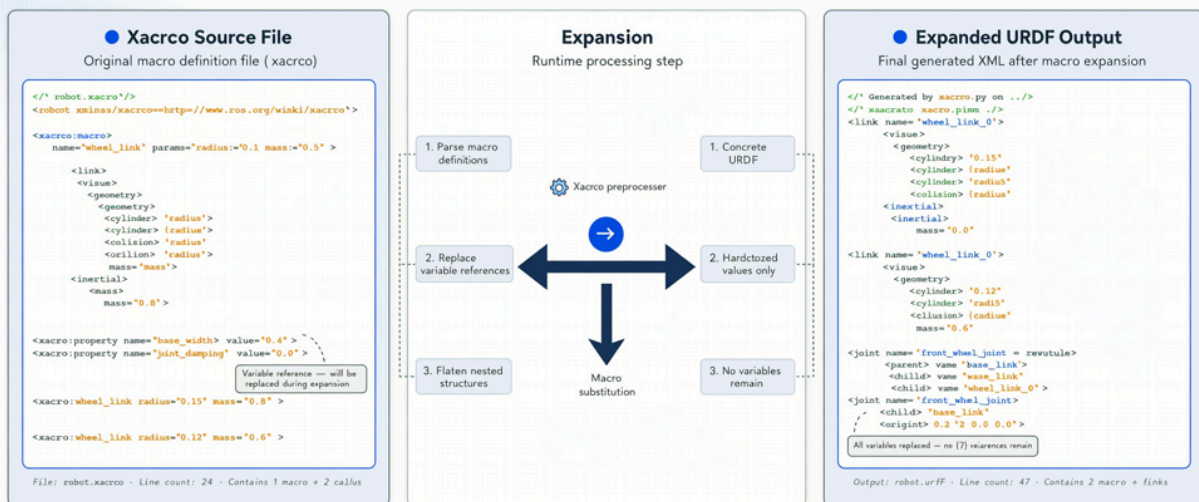
生成 .urdf (纯 XML)



robot_state_publisher 读取

Xacro Macro Expansion Process

How Xacro macros are expanded into concrete URDF at runtime



Keywords & Tags String Values & Parameters Comments & Directives Structural Elements

ROS 1 / ROS 2 Compatible · Xacro v1.14.11

\$ xacro robot.xacro > robot.urdf



维度	纯 URDF	Xacro
变量	不支持，数字硬编码	<code><xacro:property></code> 集中管理
复用	复制粘贴	<code><xacro:macro></code> 一次定义多次调用
模块化	单文件	<code><xacro:include></code> 多文件组合
数学运算	不支持	<code>\${pi/2}</code> 、 <code>\${wheel_radius*2}</code>
最终产物	直接可用	需 <code>xacro</code> 命令展开

安装 xacro 工具

```
sudo apt update
sudo apt install ros-jazzy-xacro
```

验证：

```
xacro --help
# 预期看到用法说明和选项列表
```

本节配套源码

本节源码继续使用 3.2 的 `my_robot_description` 包，并新增 Xacro 文件：

```
stage3_code/my_robot_description/
├── urdf/my_robot.urdf.xacro
├── urdf/my_robot/
│   ├── properties.xacro
│   ├── materials.xacro
│   ├── macros.xacro
│   ├── base.xacro
│   ├── sensors.xacro
│   └── wheels.xacro
├── urdf/exercise_properties.urdf.xacro
└── launch/display_xacro.launch.py
```

沿用 3.2 创建的 `~/ros2_ai_stage3_ws`；如尚未放入配套包，先执行 3.2 “使用配套源码” 中的复制步骤。本节修改 Xacro 后只需重新构建并加载环境：

```
cd ~/ros2_ai_stage3_ws
source /opt/ros/jazzy/setup.bash
colcon build --packages-select my_robot_description
source install/setup.bash
```

从 3.2 的 URDF 出发

说明： 3.2 中创建了 `diff_drive.urdf` (差速底盘) 和 `my_robot.urdf` (底盘 + 传感器支柱的入门示例；单 link 示例是 `first_box.urdf`)，二者内容不同。本节将**新建**一个独立的 `my_robot.urdf.xacro` (而非直接重命名 3.2 的某个 urdf 文件)，描述带底盘、激光雷达和轮组的完整移动机器人模型。你也可以基于 3.2 的 `diff_drive.urdf` 扩展，但建议按本节模板新建 xacro 文件以保持模块化结构。

命名约定： 3.2 使用 `left_wheel` / `right_wheel` 作为 link 名；从本节起统一为 `left_wheel_link` / `right_wheel_link` (加 `_link` 后缀)，与 TurtleBot3 和 REP-103 惯例一致。

假设我们要创建如下结构的机器人模型：

```
<?xml version="1.0"?>
<robot name="my_robot">

  <material name="blue">
    <color rgba="0.0 0.0 0.8 1.0"/>
  </material>
  <material name="black">
    <color rgba="0.0 0.0 0.0 1.0"/>
  </material>

  <!-- base_link: 底盘 -->
  <link name="base_link">
    <visual>
      <geometry>
        <box size="0.178 0.138 0.109"/>
      </geometry>
      <material name="blue"/>
    </visual>
  </link>

  <!-- base_scan: 激光雷达支架 -->
  <link name="base_scan">
    <visual>
      <geometry>
        <cylinder radius="0.05" length="0.04"/>
      </geometry>
      <material name="black"/>
    </visual>
  </link>

  <joint name="scan_joint" type="fixed">
    <parent link="base_link"/>
    <child link="base_scan"/>
    <origin xyz="0 0 0.109" rpy="0 0 0"/>
  </joint>

</robot>
```

迁移步骤

Step 1: 新建 Xacro 文件

```
cd ~/ros2_ws/src/my_robot_description/urdf
# 新建主入口文件（不要直接 mv 3.2 的 diff_drive.urdf 或 my_robot.urdf）
touch my_robot.urdf.xacro
```

Step 2: 添加 Xacro 命名空间

在 `<robot>` 根标签上声明 Xacro 命名空间——这是让解析器识别 Xacro 语法的必要步骤：

```
<?xml version="1.0"?>
<robot name="my_robot" xmlns:xacro="http://www.ros.org/wiki/xacro">
```

注意：常见旧版教程写作 `http://ros.org/wiki/xacro`。本课程统一使用 ROS 示例中更常见的 `http://www.ros.org/wiki/xacro`，避免和非 ROS 域名混淆。

Step 3: 验证迁移后的文件仍可展开

```
xacro ~/ros2_ws/src/my_robot_description/urdf/my_robot.urdf.xacro
```

如果输出合法的 URDF XML（以 `<robot name="my_robot">` 开头），说明迁移成功。此时 Xacro 文件内容与原始 URDF 等价，尚未使用任何 Xacro 特性。

文件扩展名约定

扩展名	含义
<code>.urdf</code>	纯 URDF，直接可用
<code>.urdf.xacro</code>	主机器人描述文件（最终入口）
<code>.xacro</code>	被 include 的子模块（如 <code>materials.xacro</code> 、 <code>wheel.xacro</code> ）

社区惯例：入口文件用 `.urdf.xacro`，子模块用 `.xacro`。

在 VS Code 中预览

安装 ROS 插件后，打开 `.xacro` 文件可在侧边栏预览展开后的 URDF 模型。也可手动展开后交给 `check_urdf` 校验：

```
xacro my_robot.urdf.xacro > /tmp/my_robot.urdf
check_urdf /tmp/my_robot.urdf
# 预期：Successfully parsed XML and found a robot named 'my_robot'
```

关键点

- Xacro 是 URDF 的预处理语言，增加变量、宏和模块化能力
- 迁移的第一步：重命名为 `.xacro`、添加 `xmlns:xacro` 命名空间
- 未使用 Xacro 特性前，`.xacro` 文件与原始 URDF 行为完全一致

- 使用 `xacro <file>` 命令验证文件能否正确展开
 - 入口文件推荐命名为 `*.urdf.xacro`
-

讲次 2: Xacro Properties——创建变量

知识点

Properties (属性/变量) 是 Xacro 最基础也最高频的特性。它将散落在 URDF 各处的"魔法数字"集中定义在文件顶部, 修改一处即可全局生效。

定义 Property

语法:

```
<xacro:property name="变量名" value="值"/>
```

值可以是数字、字符串, 也可以是数学表达式:

```
<!-- pi 是 xacro 内置常量, 可直接使用 ${pi/2}, 不需要自定义 -->
<xacro:property name="wheel_diameter" value="0.066"/>
<xacro:property name="wheel_radius" value="${wheel_diameter / 2.0}"/>
```

引用 Property

在 XML 属性或文本内容中用 `${变量名}` 引用:

```
<cylinder radius="${wheel_radius}" length="${wheel_width}"/>
<box size="${base_length} ${base_width} ${base_height}"/>
```

为 my_robot 添加 Properties

在 `my_robot.urdf.xacro` 的 `<robot>` 标签内、所有 Link 定义之前, 添加机器人尺寸常量:

```

<?xml version="1.0"?>
<robot name="my_robot" xmlns:xacro="http://www.ros.org/wiki/xacro">

  <!-- ===== 机器人尺寸参数（参考 TurtleBot3 Burger） ===== -->
  <xacro:property name="base_length" value="0.178"/>
  <xacro:property name="base_width" value="0.138"/>
  <xacro:property name="base_height" value="0.109"/>
  <xacro:property name="base_mass" value="0.826"/>

  <xacro:property name="wheel_radius" value="0.033"/>
  <xacro:property name="wheel_width" value="0.018"/>
  <xacro:property name="wheel_mass" value="0.0285"/>
  <xacro:property name="wheel_ygap" value="0.080"/> <!-- 轮子中心到机器人中心线的距离 -->

  <xacro:property name="scan_radius" value="0.050"/>
  <xacro:property name="scan_height" value="0.040"/>
  <xacro:property name="scan_mass" value="0.114"/>
  <xacro:property name="scan_z" value="${base_height + scan_height / 2.0}"/>

  <!-- 颜色 -->
  <xacro:property name="color_blue" value="0.0 0.0 0.8 1.0"/>
  <xacro:property name="color_black" value="0.0 0.0 0.0 1.0"/>
  <xacro:property name="color_white" value="1.0 1.0 1.0 1.0"/>

  <material name="blue">
    <color rgba="${color_blue}"/>
  </material>
  <material name="black">
    <color rgba="${color_black}"/>
  </material>

  <link name="base_link">
    <visual>
      <origin xyz="0 0 ${base_height / 2.0}" rpy="0 0 0"/>
      <geometry>
        <box size="${base_length} ${base_width} ${base_height}"/>
      </geometry>
      <material name="blue"/>
    </visual>
  </link>

  <link name="base_scan">
    <visual>
      <geometry>
        <cylinder radius="${scan_radius}" length="${scan_height}"/>
      </geometry>
      <material name="black"/>
    </visual>
  </link>

  <joint name="scan_joint" type="fixed">
    <parent link="base_link"/>
    <child link="base_scan"/>
    <origin xyz="0 0 ${scan_z}" rpy="0 0 0"/>
  </joint>

```


</robot>

Properties 支持的运算

运算	示例	结果
加减乘除	<code>\${base_height / 2}</code>	0.0545
取负	<code>\${-wheel_ygap}</code>	-0.080
引用其他变量	<code>\${wheel_radius * 2}</code>	0.066
字符串拼接	不支持复杂拼接	数字运算为主

验证展开结果

```
xacro my_robot.urdf.xacro | grep -A2 'box size'
# 预期: <box size="0.178 0.138 0.109"/>

xacro my_robot.urdf.xacro | grep 'origin xyz'
# 预期包含 base_link 的 0.0545, 以及 base_scan 的 0.129
```

Properties 的作用域

- Property 在**当前文件**中定义，对 include 进来的子文件**不可见**（除非子文件自己定义或通过参数传入）
- 后定义的 Property 可以引用先定义的 Property
- 不能在同一个文件中重复定义同名 Property

与 Launch 参数的区别

维度	Xacro Property	ROS 2 Launch Argument
解析时机	构建时 / 启动前（静态）	运行时（动态）
用途	机器人几何与物理参数	节点行为配置
修改方式	改 xacro 文件重新 build	<code>ros2 launch pkg file arg:=value</code>

后续 3.4 中，惯性质量和碰撞尺寸将直接引用这些 Properties。

关键点

- `<xacro:property name="..." value="..." />` 集中定义机器人参数
- 用 `${变量名}` 引用，支持 `${a / 2.0}` 等数学运算
- 将颜色、尺寸、质量等"魔法数字"提升到文件顶部，一处修改全局生效
- Property 是静态的，在 xacro 展开时求值，不同于 Launch 的运行时参数
- 参考 TurtleBot3 Burger 尺寸可以让后续 Gazebo 仿真更真实

讲次 3: Xacro Macros——创建宏函数

知识点

当左右轮子、多个支撑脚、阵列传感器等结构高度重复时，仅靠 Properties 仍需要大量复制粘贴。**Macros (宏)** 让你定义一个可参数化的“模板”，像函数一样多次调用。

宏的基本结构

```
<xacro:macro name="宏名称" params="参数1 参数2 默认值:=0.1">
  <!-- 宏体：可包含 link、joint、material 等任意 URDF 元素 -->
</xacro:macro>
```

调用：

```
<xacro:宏名称 参数1="值1" 参数2="值2"/>
```

定义轮子宏

在 `my_robot.urdf.xacro` 中添加：

```
<!-- ===== 宏：单个驱动轮 ===== -->
<xacro:macro name="drive_wheel" params="prefix y_reflect">
  <link name="${prefix}_wheel_link">
    <visual>
      <origin xyz="0 0 0" rpy="${pi/2} 0 0"/>
      <geometry>
        <cylinder radius="${wheel_radius}" length="${wheel_width}"/>
      </geometry>
      <material name="black"/>
    </visual>
  </link>

  <joint name="${prefix}_wheel_joint" type="continuous">
    <parent link="base_link"/>
    <child link="${prefix}_wheel_link"/>
    <origin xyz="0 ${y_reflect * wheel_ygap} 0" rpy="0 0 0"/>
    <axis xyz="0 1 0"/>
  </joint>
</xacro:macro>
```

z 偏移说明：完整模型中会有一个 `base_joint` (`base_footprint` → `base_link`，见讲次 6 综合案例) 将 `base_link` 抬升至轮心高度 (`${wheel_radius}`)，因此轮子 joint 的 z 应为 0，避免重复抬升。

`xacro` 在表达式中已经提供内置数学常量 `pi`，可直接使用 `${pi/2}`。不要再定义为 `pi` 的 property，否则 Jazzy 会提示 `redefining global symbol: pi`。

调用宏

在 `</robot>` 闭合标签之前:

```
<!-- 调用宏: 左轮 y=+0.08, 右轮 y=-0.08 -->
<xacro:drive_wheel prefix="left" y_reflect="1"/>
<xacro:drive_wheel prefix="right" y_reflect="-1"/>
```

展开后等价于手动编写 `left_wheel_link` + `left_wheel_joint` + `right_wheel_link` + `right_wheel_joint` 四套定义。

带默认值的参数

```
<xacro:macro name="box_link" params="name length width height mass:=1.0">
  <link name="${name}">
    <visual>
      <geometry>
        <box size="${length} ${width} ${height}"/>
      </geometry>
    </visual>
  </link>
</xacro:macro>

<!-- 调用时可省略 mass, 使用默认值 1.0 -->
<xacro:box_link name="payload" length="0.1" width="0.1" height="0.05"/>
```

宏内引用外部 Property

宏体可以直接使用当前文件中已定义的 Property (如 `${wheel_radius}`) , 也可以只使用传入的 params。最佳实践:

- **全局常量** (轮子半径、底盘高度) → Property
- **每次调用不同的值** (前缀名、镜像方向) → Macro 参数

验证宏展开

```
xacro my_robot.urdf.xacro | grep 'link name'
```

预期输出包含:

```
link name="base_link"
link name="base_scan"
link name="left_wheel_link"
link name="right_wheel_link"
```

```
xacro my_robot.urdf.xacro | grep 'joint name'
```

预期:

```
joint name="scan_joint"
joint name="left_wheel_joint"
joint name="right_wheel_joint"
```

宏的典型应用场景

场景	宏参数示例
左右对称轮子	prefix , y_reflect
四轮驱动	prefix , x_pos , y_pos
传感器支架	name , parent , xyz
通用碰撞体	name , geometry_type , size

关键点

- `<xacro:macro name="..." params="...">` 定义可复用模板, `<xacro:宏名 .../>` 调用
 - 宏参数支持默认值语法: `param:=default_value`
 - 宏体内用 `${param}` 引用参数, 用 `${property}` 引用全局 Property
 - 对称结构 (左右轮) 只需一个宏 + 不同参数, 避免复制粘贴
 - 用 `xacro file | grep` 检查宏是否正确展开为预期 Link/Joint
-

讲次 4: Xacro Include——在文件间引用

知识点

当机器人模型包含底盘、传感器、夹爪、外设等多个子系统时，单文件会变得难以维护。**Include** 允许将 Xacro 拆分为多个文件，按模块组织，最终由主文件组装。

Include 语法

```
<xacro:include filename="相对路径或$(find pkg)/路径"/>
```

路径相对于**当前 xacro 文件所在目录**，或使用 `$(find package_name)` 定位已安装的 ROS 包。

推荐的文件组织结构

```
my_robot_description/
├── urdf/
│   ├── my_robot.urdf.xacro    ← 主入口：组装所有模块
│   └── my_robot/
│       ├── properties.xacro   ← 全局尺寸与质量参数
│       ├── materials.xacro    ← 颜色与材质定义
│       ├── macros.xacro       ← 通用宏（轮子、支撑脚等）
│       ├── base.xacro         ← 底盘 link + base_footprint
│       ├── sensors.xacro      ← 激光雷达、相机等
│       └── wheels.xacro       ← 轮组调用
│   └── ...
├── launch/
├── rviz/
├── CMakeLists.txt
└── package.xml
```

创建子模块文件

```
urdf/my_robot/properties.xacro
```

```

<?xml version="1.0"?>
<robot xmlns:xacro="http://www.ros.org/wiki/xacro">

  <xacro:property name="base_length" value="0.178"/>
  <xacro:property name="base_width" value="0.138"/>
  <xacro:property name="base_height" value="0.109"/>
  <xacro:property name="base_mass" value="0.826"/>
  <xacro:property name="wheel_radius" value="0.033"/>
  <xacro:property name="wheel_width" value="0.018"/>
  <xacro:property name="wheel_mass" value="0.0285"/>
  <xacro:property name="wheel_ygap" value="0.080"/>
  <xacro:property name="scan_radius" value="0.050"/>
  <xacro:property name="scan_height" value="0.040"/>
  <xacro:property name="scan_mass" value="0.114"/>
  <xacro:property name="scan_z" value="${base_height + scan_height / 2.0}"/>

</robot>

```

urdf/my_robot/materials.xacro

```

<?xml version="1.0"?>
<robot xmlns:xacro="http://www.ros.org/wiki/xacro">

  <xacro:property name="color_blue" value="0.0 0.0 0.8 1.0"/>
  <xacro:property name="color_black" value="0.0 0.0 0.0 1.0"/>

  <material name="blue">
    <color rgba="${color_blue}"/>
  </material>
  <material name="black">
    <color rgba="${color_black}"/>
  </material>

</robot>

```

urdf/my_robot/macros.xacro

```
<?xml version="1.0"?>
<robot xmlns:xacro="http://www.ros.org/wiki/xacro">

  <xacro:macro name="drive_wheel" params="prefix y_reflect">
    <link name="${prefix}_wheel_link">
      <visual>
        <origin xyz="0 0 0" rpy="${pi/2} 0 0"/>
        <geometry>
          <cylinder radius="${wheel_radius}" length="${wheel_width}"/>
        </geometry>
        <material name="black"/>
      </visual>
    </link>
    <joint name="${prefix}_wheel_joint" type="continuous">
      <parent link="base_link"/>
      <child link="${prefix}_wheel_link"/>
      <origin xyz="0 ${y_reflect * wheel_ygap} 0" rpy="0 0 0"/>
      <axis xyz="0 1 0"/>
    </joint>
  </xacro:macro>

</robot>
```

主文件组装

urdf/my_robot.urdf.xacro

```
<?xml version="1.0"?>
<robot name="my_robot" xmlns:xacro="http://www.ros.org/wiki/xacro">

  <!-- 按依赖顺序 include: 先属性, 再材质, 再宏, 最后组件 -->
  <xacro:include filename="my_robot/properties.xacro"/>
  <xacro:include filename="my_robot/materials.xacro"/>
  <xacro:include filename="my_robot/macros.xacro"/>
  <xacro:include filename="my_robot/base.xacro"/>
  <xacro:include filename="my_robot/sensors.xacro"/>
  <xacro:include filename="my_robot/wheels.xacro"/>

</robot>
```

Include 顺序原则

```
properties.xacro → 全局变量, 必须最先
materials.xacro  → 材质 (可引用 properties 中的颜色变量)
macros.xacro     → 宏定义 (引用 properties)
base/sensors/wheels → 定义 link/joint 或调用宏
主文件          → 组装一切
```

使用 \$(find ...) 引用已安装包

当 xacro 文件已安装到 share/ 后, 可用 ROS 包路径:


```
<xacro:include filename="$(find my_robot_description)/urdf/my_robot/properties.xacro"/>
```

这在 Launch 文件通过 `xacro` 处理时尤为重要 (3.4 将详细讲解)。

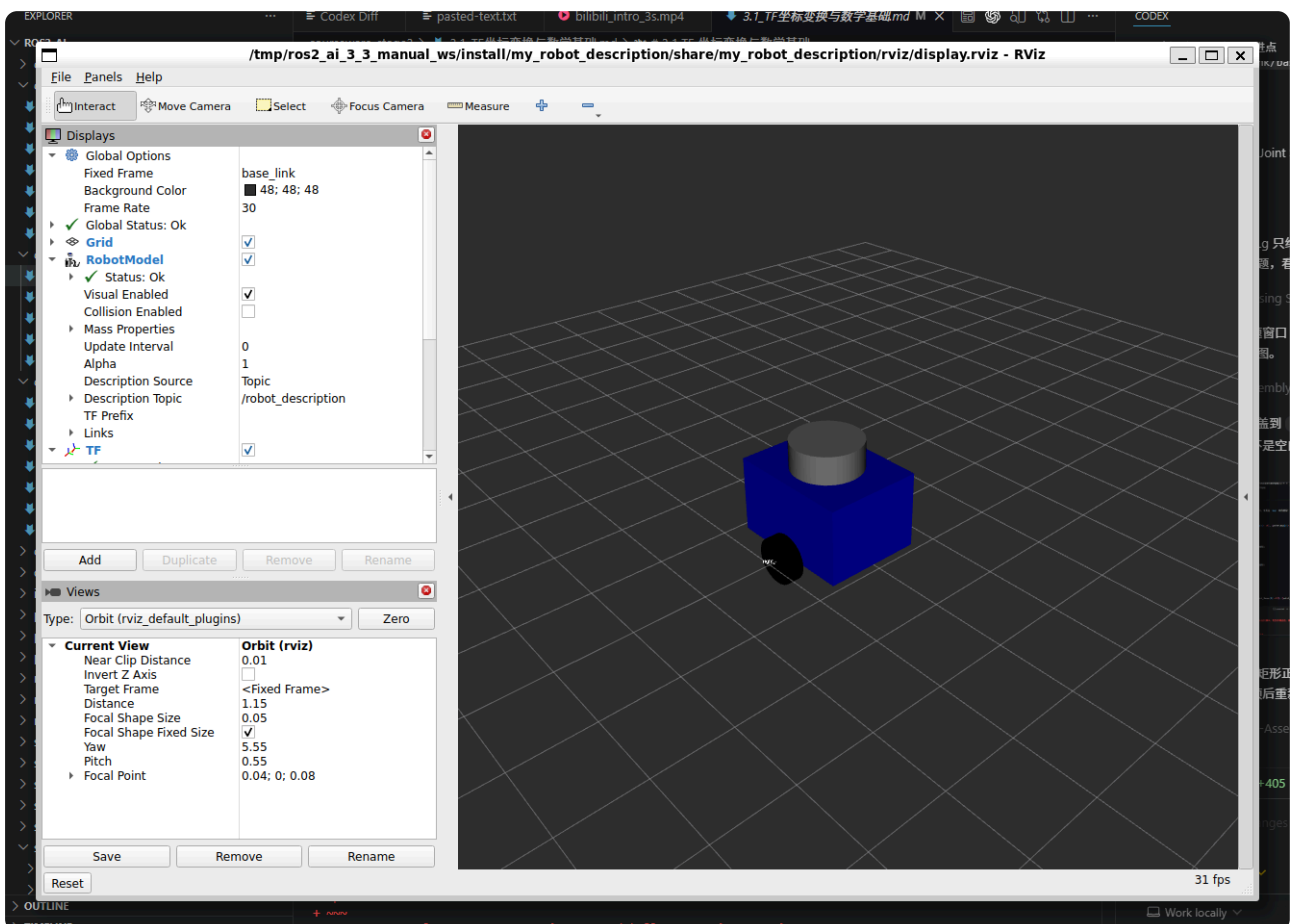
运行本节配套模型

完成构建并 `source install/setup.bash` 后, 可以按下面的顺序展开、校验并启动 RViz:

```
xacro $(ros2 pkg prefix my_robot_description)/share/my_robot_description/urdf/my_robot.urdf.xacro
check_urdf /tmp/my_robot_from_xacro.urdf
urdf_to_graphviz /tmp/my_robot_from_xacro.urdf
ros2 launch my_robot_description display_xacro.launch.py
```

版本说明: 本讲代码片段搭建的是 4 link / 3 joint 的中间版本 (`base_link`、`base_scan`、左右轮)。仓库中安装的 `my_robot.urdf.xacro` 是 3.4~3.5 完成后的完整版本 (6 link / 5 joint, 额外含 `base_footprint`、`back_caster_link`, 以及惯性、碰撞和 Gazebo 材质模块), 因此 `check_urdf` 会显示更多 link/joint, 属于正常现象。

真实运行效果如下, RViz 中应能看到蓝色底盘、黑色左右驱动轮和顶部雷达; `tf2_echo base_link base_scan` 的 z 方向平移应为 `0.129`, 对应 `${base_height} + scan_height / 2.0`。



关键点

- `<xacro:include filename="..." />` 将模型拆分为多文件模块
 - 子文件也需要 `xmlns:xacro` 命名空间声明
 - Include 顺序: Properties → Materials → Macros → 组件
 - 主文件 `*.urdf.xacro` 负责组装, 子模块放在子目录 (如 `my_robot/`)
 - 模块化是大型机器人项目的标准工程实践
-

讲次 5: Xacro 命令生成 URDF

知识点

Xacro 文件不能直接被 `robot_state_publisher` 或 RViz 使用——必须先**展开**为纯 URDF。本节学习命令行工具用法，以及如何将 xacro 处理集成到 ROS 2 构建和 Launch 流程。

基本命令

```
# 展开到终端（检查用）
xacro my_robot.urdf.xacro

# 展开并保存为 URDF 文件
xacro my_robot.urdf.xacro -o my_robot.urdf

# 等价写法（重定向）
xacro my_robot.urdf.xacro > my_robot.urdf
```

常用选项

选项	说明	示例
<code>-o FILE</code>	输出到文件	<code>xacro model.xacro -o model.urdf</code>
<code>--inorder</code>	按文档顺序处理（默认行为）	处理有依赖顺序的宏
<code>-q</code>	静默模式，减少日志	用于脚本
<code>arg:=value</code>	传入 xacro 参数（高级）	<code>xacro model.xacro wheel_radius:=0.05</code>

使用 `arg:=value` 前，需在 xacro 文件中声明参数：

```
<xacro:arg name="wheel_radius" default="0.033"/>
<!-- 然后在 property 中引用: -->
<xacro:property name="wheel_radius" value="$(arg wheel_radius)"/>
```

校验生成的 URDF

```
# 语法与结构检查
check_urdf my_robot.urdf

# 可视化 TF 树
urdf_to_graphviz my_robot.urdf
# 生成 my_robot.pdf（可用 evince 或浏览器打开）
```

`check_urdf` 输出示例：

```
robot name is: my_robot
----- Successfully parsed XML and urdf to robot model -----
```

在 Launch 文件中动态展开（推荐方式）

ROS 2 项目**不推荐**将 URDF 预生成并提交到 Git，而是在 Launch 时动态调用 xacro。3.4 将创建完整 Launch 文件，此处先了解核心模式：

```
import os
from ament_index_python.packages import get_package_share_directory
from launch import LaunchDescription
from launch_ros.actions import Node
import xacro

def generate_launch_description():
    pkg_path = get_package_share_directory('my_robot_description')
    xacro_file = os.path.join(pkg_path, 'urdf', 'my_robot.urdf.xacro')

    # 展开 xacro 为 URDF 字符串
    robot_description = xacro.process_file(xacro_file).toxml()

    return LaunchDescription([
        Node(
            package='robot_state_publisher',
            executable='robot_state_publisher',
            parameters=[{'robot_description': robot_description}],
        ),
    ])
```

在 CMakeLists.txt 中安装 urdf 目录

```
install(DIRECTORY urdf
        DESTINATION share/${PROJECT_NAME}
)
```

在 package.xml 中声明依赖

```
<depend>xacro</depend>
<depend>robot_state_publisher</depend>
<depend>rviz2</depend>
```

完整工作流



常见错误排查

错误信息	原因	解决
Invalid parameter "..."	宏调用缺少必要参数	检查 <code><xacro:macro params="..."></code>
property 'xxx' undefined	引用未定义的 Property	确认 include 顺序
No such file: ...xacro	include 路径错误	检查相对路径或 <code>\$(find pkg)</code>
XML syntax error	标签未闭合或特殊字符	检查 <code>\${}</code> 表达式

关键点

- `xacro file.xacro -o file.urdf` 将 Xacro 展开为纯 URDF
 - 开发时用 `check_urdf` 和 `urdf_to_graphviz` 验证模型
 - 生产环境在 Launch 文件中用 `xacro.process_file()` 动态展开，不提交生成的 URDF
 - `CMakeLists.txt` 需 `install(DIRECTORY urdf ...)` 安装 xacro 文件
 - `package.xml` 声明 `xacro` 依赖
-

讲次 6: 【Activity】Xacro Properties 练习与解答

知识点

练习目标

在不使用宏的前提下，仅通过 **Xacro Properties** 完成一个参数化的双轮小车模型。完成后应能用 `xacro` 命令展开并通过 `check_urdf` 校验。

练习要求

- 创建文件 `~/ros2_ws/src/my_robot_description/urdf/exercise_properties.urdf.xacro`
- 在文件顶部用 Properties 定义以下参数（参考 TurtleBot3 Burger）：

参数名	值	说明
<code>chassis_length</code>	0.178	底盘长度 (m)
<code>chassis_width</code>	0.138	底盘宽度 (m)
<code>chassis_height</code>	0.109	底盘高度 (m)
<code>wheel_radius</code>	0.033	轮子半径 (m)
<code>wheel_width</code>	0.018	轮子宽度 (m)
<code>wheel_offset_y</code>	0.080	轮子中心到车体中心线距离 (m)
<code>wheel_offset_z</code>	0.033	轮子中心高度 (= wheel_radius)

- 创建以下 Link 和 Joint（所有数值必须通过 `${}` 引用 Properties，不允许硬编码）：
- `base_footprint` → `base_link`（fixed joint, z 偏移 `${wheel_offset_z}`）
- `base_link`：visual 为 box，尺寸引用 chassis 三个参数
- `left_wheel_link` + `left_wheel_joint`（continuous, y = `+${wheel_offset_y}`）
- `right_wheel_link` + `right_wheel_joint`（continuous, y = `-${wheel_offset_y}`）
- 每个轮子 visual 为 cylinder，绕 x 轴旋转 90°（`rpv="${pi/2} 0 0"`）
- 定义蓝色和黑色两种 material，颜色也通过 Property 定义

验证命令

```
cd ~/ros2_ws
xacro src/my_robot_description/urdf/exercise_properties.urdf.xacro -o /tmp/exercise.urdf
check_urdf /tmp/exercise.urdf
urdf_to_graphviz /tmp/exercise.urdf
```

参考答案

exercise_properties.urdf.xacro

```

<?xml version="1.0"?>
<robot name="exercise_robot" xmlns:xacro="http://www.ros.org/wiki/xacro">

  <!-- Properties -->
  <xacro:property name="chassis_length" value="0.178"/>
  <xacro:property name="chassis_width" value="0.138"/>
  <xacro:property name="chassis_height" value="0.109"/>
  <xacro:property name="wheel_radius" value="0.033"/>
  <xacro:property name="wheel_width" value="0.018"/>
  <xacro:property name="wheel_offset_y" value="0.080"/>
  <xacro:property name="wheel_offset_z" value="${wheel_radius}"/>

  <xacro:property name="color_blue" value="0.0 0.0 0.8 1.0"/>
  <xacro:property name="color_black" value="0.0 0.0 0.0 1.0"/>

  <material name="blue">
    <color rgba="${color_blue}"/>
  </material>
  <material name="black">
    <color rgba="${color_black}"/>
  </material>

  <!-- base_footprint -->
  <link name="base_footprint"/>

  <link name="base_link">
    <visual>
      <origin xyz="0 0 ${chassis_height / 2.0}" rpy="0 0 0"/>
      <geometry>
        <box size="${chassis_length} ${chassis_width} ${chassis_height}"/>
      </geometry>
      <material name="blue"/>
    </visual>
  </link>

  <joint name="base_joint" type="fixed">
    <parent link="base_footprint"/>
    <child link="base_link"/>
    <origin xyz="0 0 ${wheel_offset_z}" rpy="0 0 0"/>
  </joint>

  <!-- 左轮 -->
  <link name="left_wheel_link">
    <visual>
      <origin xyz="0 0 0" rpy="${pi/2} 0 0"/>
      <geometry>
        <cylinder radius="${wheel_radius}" length="${wheel_width}"/>
      </geometry>
      <material name="black"/>
    </visual>
  </link>

  <joint name="left_wheel_joint" type="continuous">
    <parent link="base_link"/>
    <child link="left_wheel_link"/>
    <origin xyz="0 ${wheel_offset_y} 0" rpy="0 0 0"/>
    <axis xyz="0 1 0"/>
  </joint>

```



```

</joint>

<!-- 右轮 -->
<link name="right_wheel_link">
  <visual>
    <origin xyz="0 0 0" rpy="{pi/2} 0 0"/>
    <geometry>
      <cylinder radius="{wheel_radius}" length="{wheel_width}"/>
    </geometry>
    <material name="black"/>
  </visual>
</link>

<joint name="right_wheel_joint" type="continuous">
  <parent link="base_link"/>
  <child link="right_wheel_link"/>
  <origin xyz="0 {-wheel_offset_y} 0" rpy="0 0 0"/>
  <axis xyz="0 1 0"/>
</joint>

</robot>

```

自检清单

- [] 文件中无任何硬编码尺寸数字 (含 origin、box size、cylinder radius)
- [] `check_urdf` 输出 `Successfully parsed XML`
- [] TF 树: `base_footprint` → `base_link` → `left/right_wheel_link`
- [] 修改 `wheel_radius` 为 `0.05` 后重新展开, 轮子尺寸自动更新

关键点

- 练习核心: 用 Properties 消除所有硬编码, 验证"改一处、全局生效"
 - `base_footprint` 是移动机器人的标准根坐标系, z 偏移等于轮半径
 - 轮子 joint 的 origin 在 `base_link` 坐标系下, y 方向用 `{wheel_offset_y}` 和 `{-wheel_offset_y}` 对称
 - continuous 关节的 axis 为 `0 1 0` (绕 y 轴旋转, 符合差速驱动模型)
 - 通过 `check_urdf` + `urdf_to_graphviz` 验证模型结构正确
-

本节总结

命令速查表

命令	说明
<code>xacro file.xacro</code>	展开 Xacro 到终端
<code>xacro file.xacro -o out.urdf</code>	展开并保存 URDF
<code>check_urdf file.urdf</code>	校验 URDF 结构
<code>urdf_to_graphviz file.urdf</code>	生成 TF 树 PDF

Xacro 语法速查

语法	用途
<code>xmlns:xacro="http://www.ros.org/wiki/xacro"</code>	声明命名空间
<code><xacro:property name="k" value="v"/></code>	定义变量
<code>\${name} / \${a/2}</code>	引用变量 / 运算
<code><xacro:macro name="m" params="a b"></code>	定义宏
<code><xacro:m a="1" b="2"/></code>	调用宏
<code><xacro:include filename="path"/></code>	引入子文件

下一步

下一节 **3.4 仿真准备与机器人描述包** 中，我们将为 Xacro 模型添加 **Collision** 和 **Inertial** 标签（Gazebo Harmonic 物理仿真的必要前提），修复惯性参数，配置 Gazebo 材质颜色，并创建完整的 `my_robot_description` 功能包与 Launch 文件，实现一键在 RViz 2 中显示机器人模型。